

Spec 03 — Autorização & privacidade

1. Matriz de permissões

Actor	Ler lista pública	RSVP	Reservar	Cancelar	CRUD presente	Ver convidados	Ver quem reservou
Anônimo (sem token)	✗	✗	✗	✗	✗	✗	✗
Convidado (tem <code>listToken</code>)	✓	✓	✓	própria	✗	✗	✗
Host (autenticado)	✓	✓	✓	qualquer	✓	✓	✓ (salvo surpresa)

2. Token de compartilhamento

```
// src/entities/registry/token.ts (já existente – correto)
export function generateListToken(): string {
  const bytes = new Uint8Array(16); // 128 bits
  crypto.getRandomValues(bytes); // CSPRNG
  return btoa(String.fromCharCode(...bytes))
    .replace(/\+/g, "-").replace(/\//g, "_").replace(/=/g, ""); // base64url ~22 chars
}
```

Nunca derivar de id/data/nome. **Rate limit** por IP no `GET /registry/:token` contra varredura. O token de seed `festa-rodriigo-25` é só mock — **não vai para produção**.

3. Modo "surpresa"

Quando `events.surprise_mode = true`:

- `GET /registry/:token` **não inclui** `guestName` nas reservas (só `{ giftId, reservedAt }`).
- Dashboard do host omite identidade do reservante (só contagem / valor arrecadado).
- `reservation_log` mantém histórico completo, **nunca** exposto na API pública.

Implementar na **camada de resposta** da função/endpoint (simples e auditável), não com RLS complexa.

4. Supabase — RLS (recomendado p/ MVP)

Escritas de convidado passam por Edge Function (gateway); o cliente anon nunca escreve direto.

```
cliente (anon key)
├ GET rpc('get_registry', { list_token }) → SECURITY DEFINER fn
├ POST functions.invoke('reserve-gift', ...) → Edge Function (service_role)
├ POST functions.invoke('submit-rsvp', ...)
└ POST functions.invoke('post-message', ...)
```

```

alter table events enable row level security;
create policy "host manages own event" on events for all to authenticated
  using (auth.uid() = host_id) with check (auth.uid() = host_id);

alter table gifts enable row level security;
create policy "host manages own gifts" on gifts for all to authenticated
  using (event_id in (select id from events where host_id = auth.uid()))
  with check (event_id in (select id from events where host_id = auth.uid()));

alter table reservations enable row level security;
create policy "host reads own event reservations" on reservations for select to authenticated
  using (event_id in (select id from events where host_id = auth.uid()));
-- escrita só via service_role (Edge Function), que bypassa RLS

-- guests, rsvps, messages: mesma lógica (host-only para leitura de PII)

```

Use `(select auth.uid())` nas policies (avaliação única por query, evita oracle por linha).

5. Backend dedicado — middleware (alternativa)

```

async function requireListToken(req, res, next) {
  const token = req.params.token ?? req.body.listToken;
  if (!token) return res.status(401).json({ code: 'MISSING_TOKEN' });
  const event = await db.query('select id, surprise_mode, status from events where list_token = $1', [token]);
  if (!event) return res.status(404).json({ code: 'LIST_NOT_FOUND' });
  if (event.status === 'draft') return res.status(403).json({ code: 'LIST_NOT_ACTIVE' });
  req.eventContext = { eventId: event.id, surpriseMode: event.surprise_mode };
  next();
}

async function requireHost(req, res, next) {
  const payload = await verifyJwt(req.headers.authorization?.replace('Bearer ', ''));
  const event = await db.query('select id from events where id=$1 and host_id=$2', [req.params.eventId, payload.sub]);
  if (!event) return res.status(403).json({ code: 'NOT_HOST' });
  next();
}

```

Trade-off: RLS garante isolamento mesmo com a chave publishable pública, mas divide a authz entre SQL e código (mais difícil de testar unitariamente). Backend dedicado concentra a authz em código testável, mas você opera a infra.